

Day 7 — Writing a Testbench and Simulating

A testbench is a Verilog file that tests another Verilog module. It has no ports — it's a self-contained test program that applies inputs and observes outputs. You wrote **tb_decoder.v** by hand today and compiled it with iverilog.

```
module tb_decoder;
  reg [3:0] opcode; // you drive this
  wire [2:0] alu_op; // decoder drives this
  wire reg_write;
  wire mem_read;
  wire mem_write;
  wire branch;
  wire jump;
  wire use_imm;

  decoder uut( // instantiate decoder
    .opcode(opcode),
    .alu_op(alu_op),
    .reg_write(reg_write),
    // ...
  );

  initial begin
    $display("Testing CUSTOM16 Decoder!");
    opcode = 4'b0000; #10; // ADD
    $display("ADD: alu_op=%b reg_write=%b ...", alu_op, reg_write, ...);
    // ... repeat for all 14 instructions ...
    $display("Simulation complete.");
    $finish;
  end

endmodule
```

Simulation commands:

```
# Compile both files together
iverilog -o sim.vvp decoder.v tb_decoder.v

# Run the simulation
vvp sim.vvp
```

What iverilog and vvp do:

iverilog is the compiler — takes your Verilog files and produces a simulation executable (sim.vvp). vvp is the simulator — runs that executable and produces the output. Same concept as compile then execute in C.

Day 8 — Auto-Generating the Testbench

You extended **emitter.py** with an **emit_testbench()** method that automatically generates **tb_decoder.v** from the ISA spec. The hand-written testbench is now obsolete — your Python writes it.

```
def emit_testbench(self):
    lines = []
    lines.append('module tb_decoder;')
    lines.append(' reg[' + str(self.spec.opcode_bits-1) + ':0] opcode;')

    for sig in self.spec.ControlSignals:
        if sig.width > 1:
            lines.append(' wire[' + str(sig.width-1) + ':0] ' + sig.name + ';')
        else:
            lines.append(' wire ' + sig.name + ';')

    # decoder instantiation
    lines.append(' decoder uut(')
    lines.append(' .opcode(opcode),')
    for i, sig in enumerate(self.spec.ControlSignals):
        comma = '' if i == len(self.spec.ControlSignals)-1 else ','
        lines.append(' .' + sig.name + '(' + sig.name + ')' + comma)
    lines.append(' );')

    # initial block
    lines.append(' initial begin')
    for instr in self.spec.Instructions:
        lines.append(' opcode = 4\'b' + instr.opcode + '; #10; // ' + instr.mnemonic)
        lines.append(' $display("'" + instr.mnemonic + ': ...", ...);')
    lines.append(' $finish;')
    lines.append(' end')
    lines.append(' endmodule')
```

Key Concepts Learned — Days 7 & 8

Concept	What it does
<code>module tb_decoder;</code>	Testbench module — no ports, self-contained
<code>reg [3:0] opcode;</code>	Signal you control — drive it in the testbench
<code>wire [2:0] alu_op;</code>	Signal driven by decoder — you observe it
<code>decoder uut(...)</code>	Instantiate decoder — connect testbench signals to ports
<code>.port(signal)</code>	Port connection syntax — <code>.decoder_port(testbench_signal)</code>
<code>initial begin...end</code>	Runs once at start of simulation
<code>#10</code>	Wait 10 time units — give decoder time to respond
<code>\$display(...)</code>	Print to terminal — like Python's <code>print()</code>
<code>%b</code>	Format specifier — print value in binary
<code>\$finish</code>	Stop the simulator
<code>iverilog -o sim.vvp a.v b.v</code>	Compile Verilog files into simulation executable
<code>vvp sim.vvp</code>	Run the simulation executable

Verified Simulation Output

```
Testing CUSTOM16 Decoder!
ADD : alu_op=000 reg_write=1 mem_read=0 mem_write=0 branch=0 jump=0 use_imm=0
SUB : alu_op=001 reg_write=1 mem_read=0 mem_write=0 branch=0 jump=0 use_imm=0
LOAD : alu_op=111 reg_write=1 mem_read=1 mem_write=0 branch=0 jump=0 use_imm=1
STORE: alu_op=111 reg_write=0 mem_read=0 mem_write=1 branch=0 jump=0 use_imm=1
BEQ : alu_op=001 reg_write=0 mem_read=0 mem_write=0 branch=1 jump=0 use_imm=1
JUMP : alu_op=111 reg_write=0 mem_read=0 mem_write=0 branch=0 jump=1 use_imm=1
HALT : alu_op=111 reg_write=0 mem_read=0 mem_write=0 branch=0 jump=0 use_imm=0
Simulation complete.
```

Toolchain Progress After Day 8

✓ isa_spec.json — 14 instructions, 7 control signals, single source of truth ■

- ✓ parser.py — reads JSON, builds Python objects ■
- ✓ validator.py — catches 4 categories of spec errors ■
- ✓ emitter.py — generates decoder.v and tb_decoder.v automatically ■
- ✓ decoder.v — auto-generated synthesizable Verilog ■
- ✓ tb_decoder.v — auto-generated testbench ■
- ✓ Simulation — all 14 instructions verified correct ■

■ Homework — Before Day 9

- *Tomorrow is Day 9 — ROM-based control unit*
- *Think about this: what is the difference between a case statement and a lookup table?*
- *Your decoder uses a case statement. A ROM uses a memory array indexed by opcode.*
- *Both do the same thing — opcode in, control signals out. Different hardware implementation.*